

Fractal Explorer

Documentación Técnica del Proyecto

Autor: Daniel Vega Miranda

Fecha: Enero 2026

Versión: 1.0.0

Repositorio: <https://github.com/DaniVegaM/fractal-explorer>

Tabla de Contenidos

1. [Introducción](#)
 2. [Objetivos del Proyecto](#)
 3. [Marco Teórico](#)
 4. [Arquitectura del Sistema](#)
 5. [Tecnologías Utilizadas](#)
 6. [Estructura del Proyecto](#)
 7. [Componentes del Sistema](#)
 8. [Algoritmos Implementados](#)
 9. [Interfaz de Usuario](#)
 10. [Manual de Instalación](#)
 11. [Manual de Usuario](#)
 12. [Optimizaciones de Rendimiento](#)
 13. [Conclusiones](#)
 14. [Referencias](#)
-

1. Introducción

Fractal Explorer es una aplicación web interactiva desarrollada con tecnologías modernas de frontend que permite a los usuarios explorar y visualizar fractales matemáticos en tiempo real. La aplicación ofrece una experiencia inmersiva donde el usuario puede navegar, hacer zoom y personalizar la visualización de diferentes tipos de fractales directamente en su navegador.

Los fractales son estructuras matemáticas que exhiben auto-similitud a diferentes escalas, lo que significa que al hacer zoom en cualquier parte del fractal, se revelan patrones similares al todo. Esta propiedad los hace infinitamente detallados y visualmente fascinantes.

1.1 Problemática

La visualización de fractales requiere cálculos matemáticos intensivos que, tradicionalmente, requerían software especializado de escritorio. Con el avance de las tecnologías web, es posible crear experiencias interactivas que permitan a cualquier persona explorar estos fascinantes objetos matemáticos directamente en su navegador.

1.2 Solución Propuesta

El proyecto Fractal Explorer proporciona una solución web moderna que utiliza Web Workers para realizar cálculos en paralelo, Canvas API para renderizado eficiente, y una interfaz de usuario intuitiva construida con React y TailwindCSS.

2. Objetivos del Proyecto

2.1 Objetivo General

Desarrollar una aplicación web interactiva que permita a los usuarios explorar y visualizar fractales matemáticos en tiempo real, con controles intuitivos y múltiples opciones de personalización.

2.2 Objetivos Específicos

1. **Implementar múltiples tipos de fractales:** Mandelbrot, Julia, Burning Ship y Tricorn.
2. **Optimizar el rendimiento:** Utilizar Web Workers para cálculos paralelos y técnicas de renderizado progresivo.
3. **Proporcionar controles interactivos:** Zoom, navegación por arrastre, y ajuste de parámetros en tiempo real.

4. **Ofrecer personalización visual:** Múltiples paletas de colores y parámetros ajustables.
 5. **Permitir exportación:** Descarga de imágenes en formato PNG.
 6. **Asegurar compatibilidad:** Funcionamiento en navegadores modernos con diseño responsive.
-

3. Marco Teórico

3.1 ¿Qué es un Fractal?

Un fractal es un objeto geométrico cuya estructura básica, fragmentada o irregular, se repite a diferentes escalas. El término fue acuñado por el matemático Benoît Mandelbrot en 1975, derivado del latín *fractus*, que significa “quebrado” o “fracturado”.

3.2 Conjunto de Mandelbrot

El conjunto de Mandelbrot es el fractal más famoso y se define matemáticamente como el conjunto de números complejos c para los cuales la función:

$$z_{n+1} = z_n^2 + c$$

no diverge al infinito cuando se itera desde $z_0 = 0$.

Algoritmo de escape:

- Para cada punto c en el plano complejo
- Iterar $z_{n+1} = z_n^2 + c$ comenzando con $z_0 = 0$
- Si $|z_n| > 2$ (radio de escape), el punto escapa
- Si después de N iteraciones el punto no ha escapado, se considera parte del conjunto

3.3 Conjunto de Julia

Los conjuntos de Julia están estrechamente relacionados con el conjunto de Mandelbrot. Para un valor fijo de c , el conjunto de Julia es el conjunto de puntos iniciales z_0 para los cuales la iteración:

$$z_{n+1} = z_n^2 + c$$

no diverge al infinito. A diferencia del Mandelbrot, aquí c es constante y z_0 varía según la posición del pixel.

3.4 Burning Ship

El fractal Burning Ship es una variación del Mandelbrot que utiliza valores absolutos:

$$z_{n+1} = (|Re(z_n)| + i|Im(z_n)|)^2 + c$$

Esto produce una forma que recuerda a un barco en llamas, de ahí su nombre.

3.5 Tricorn (Mandelbar)

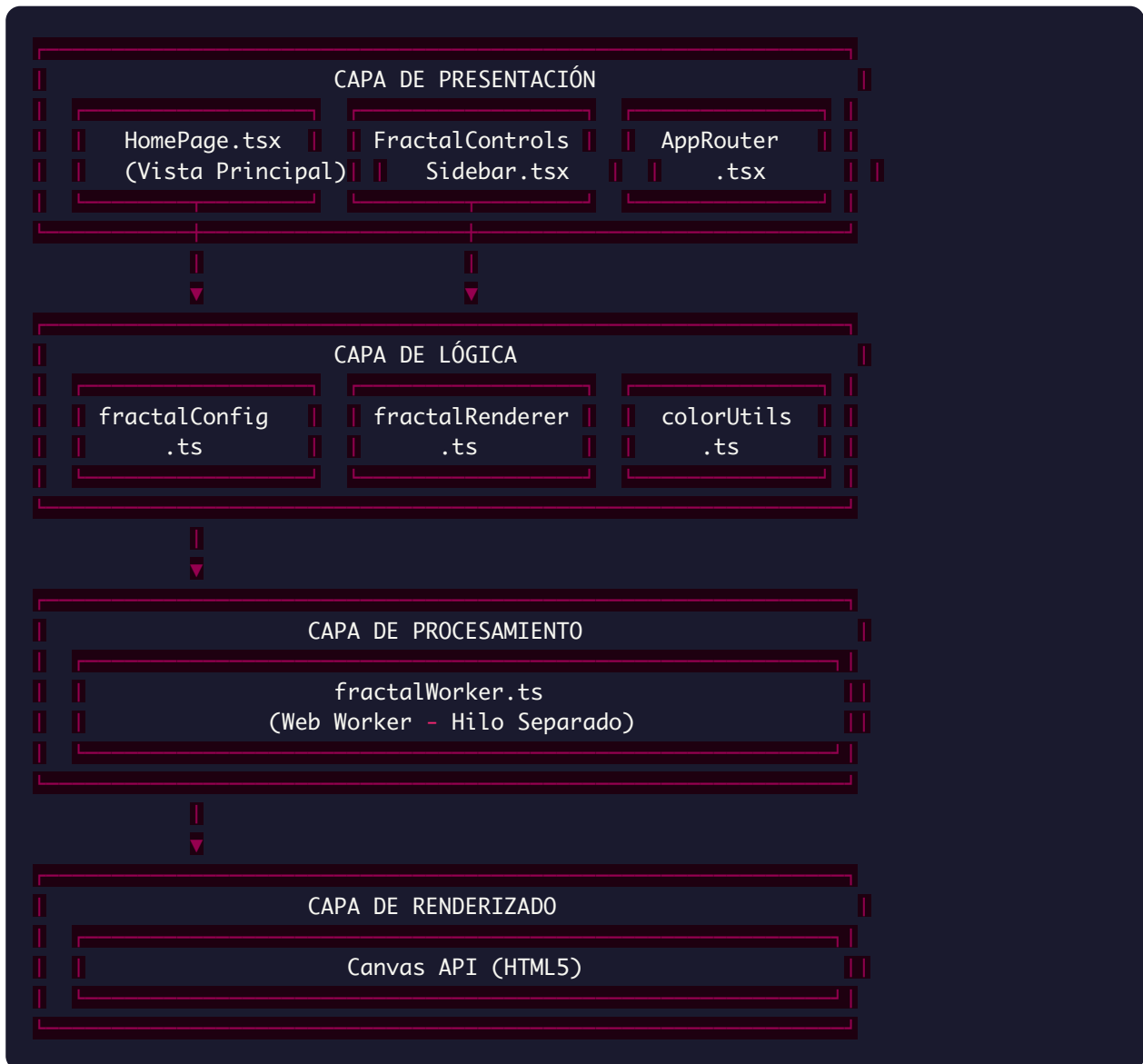
El Tricorn, también conocido como Mandelbar, utiliza el conjugado complejo:

$$z_{n+1} = \bar{z}_n^2 + c$$

Donde $\bar{z}$ representa el conjugado complejo de z . Esto produce formas características de “cuernos”.

4. Arquitectura del Sistema

4.1 Diagrama de Arquitectura



4.2 Flujo de Datos

- Entrada del Usuario:** El usuario interactúa con la interfaz (zoom, arrastrar, cambiar parámetros)
- Gestión de Estado:** React gestiona el estado de la aplicación (parámetros, centro, paleta)
- Envío al Worker:** Los parámetros se envían al Web Worker para cálculo
- Cálculo del Fractal:** El Worker calcula los valores de cada pixel
- Retorno de Datos:** El Worker devuelve los datos de imagen calculados
- Renderizado:** Los datos se dibujan en el Canvas

5. Tecnologías Utilizadas

5.1 Stack Principal

Tecnología	Versión	Propósito
React	19.1.1	Biblioteca de UI para construir interfaces
TypeScript	~5.9.3	Superset tipado de JavaScript
Vite	7.1.7	Herramienta de build y desarrollo
TailwindCSS	4.1.16	Framework CSS utilitario
React Router	7.9.4	Enrutamiento para SPA
Lucide React	0.548.0	Biblioteca de íconos

5.2 Herramientas de Desarrollo

Herramienta	Versión	Propósito
ESLint	9.36.0	Linter para JavaScript/TypeScript
TypeScript ESLint	8.45.0	Reglas de ESLint para TypeScript
Vite Plugin React SWC	4.1.0	Compilación rápida con SWC

5.3 APIs del Navegador

- **Canvas API:** Para renderizado 2D de los fractales
- **Web Workers API:** Para cálculos en hilos separados
- **ImageData API:** Para manipulación de datos de imagen a nivel de pixel

6. Estructura del Proyecto

```
FractalExplorer/
├── public/           # Archivos estáticos públicos
├── src/
│   ├── assets/      # Recursos (imágenes, fuentes)
│   ├── components/
│   │   └── FractalControlsSidebar.tsx # Panel de controles
│   ├── lib/
│   │   ├── colorUtils.ts  # Utilidades de color y paletas
│   │   ├── fractalConfig.ts # Configuración de fractales
│   │   ├── fractalRenderer.ts # Renderizador principal (fallback)
│   │   └── fractalWorker.ts # Web Worker para cálculos
│   ├── pages/
│   │   └── HomePage.tsx    # Página principal
│   ├── router/
│   │   └── AppRouter.tsx   # Configuración de rutas
│   ├── App.tsx           # Componente raíz
│   ├── index.css         # Estilos globales
│   └── main.tsx          # Punto de entrada
├── docs/                # Documentación del proyecto
├── eslint.config.js     # Configuración de ESLint
├── index.html           # HTML principal
├── package.json         # Dependencias y scripts
├── tsconfig.json        # Configuración de TypeScript
├── tsconfig.app.json    # Config TS para la app
├── tsconfig.node.json   # Config TS para Node
└── vite.config.ts       # Configuración de Vite
```

7. Componentes del Sistema

7.1 App.tsx (Componente Raíz)

```
import { AppRouter } from '../router/AppRouter'

function App() {
  return (
    <AppRouter />
  )
}

export default App
```

El componente raíz de la aplicación que simplemente renderiza el enrutador.

7.2 AppRouter.tsx (Enrutador)

```
import { createBrowserRouter, RouterProvider } from "react-router";
import HomePage from "../pages/HomePage";

const router = createBrowserRouter([
  {
    path: "/",
    element: <HomePage />,
  },
]);

export const AppRouter = () => {
  return <RouterProvider router={router} />;
};
```

Configuración del enrutador utilizando React Router v7 con la nueva API.

7.3 HomePage.tsx (Página Principal)

Este es el componente central que contiene:

- **Estado de la aplicación:**
 - `selectedFractal` : Tipo de fractal seleccionado
 - `params` : Parámetros del fractal (iteraciones, zoom, etc.)
 - `center` : Coordenadas del centro de la vista
 - `selectedPalette` : Paleta de colores activa
 - `showInfo` , `showControls` : Controles de UI
- **Referencias (Refs):**
 - `canvasRef` : Referencia al elemento Canvas
 - `workerRef` : Referencia al Web Worker
 - `isDragging` , `lastMousePos` : Para manejo de arrastre
- **Funciones principales:**
 - `renderToCanvas()` : Renderiza el fractal en el canvas
 - `handleZoom()` : Maneja el zoom in/out
 - `handleMouseMove()` : Maneja el arrastre para navegación
 - `handleDownload()` : Exporta la imagen actual

7.4 FractalControlsSidebar.tsx (Panel de Controles)

Componente de UI que proporciona:

- **Selector de Fractal:** Botones para cambiar entre tipos de fractal
- **Controles de Parámetros:** Sliders para ajustar iteraciones, zoom, etc.
- **Paletas de Colores:** Grid de botones para seleccionar paleta
- **Controles de Navegación:** Botones de zoom y reset

7.5 fractalConfig.ts (Configuración)

Define los tipos y configuraciones:

```
export type FractalType = "mandelbrot" | "julia" | "burning-ship" | "tricorn";

export type PaletteType =
  | 'grayscale' | 'fire' | 'ocean' | 'cosmic'
  | 'rainbow' | 'neon' | 'earth' | 'gradient';

export interface FractalParam {
  label: string;
  key: string;
  min: number;
  max: number;
  step: number;
  default: number;
  description: string;
}

export interface FractalConfig {
  name: string;
  description: string;
  params: FractalParam[];
  defaultCenter: { x: number; y: number };
}
```

7.6 fractalWorker.ts (Web Worker)

El Web Worker es el corazón computacional de la aplicación. Ejecuta los cálculos de fractal en un hilo separado para no bloquear la UI.

Características principales:

- Caché de paletas de colores (lookup tables)
- Funciones optimizadas para cada tipo de fractal
- Comunicación mediante mensajes con el hilo principal

7.7 colorUtils.ts (Utilidades de Color)

Funciones para generar colores:

```
// Interpolación lineal
export function lerp(a: number, b: number, t: number): number

// Conversión HSL a RGB
export function hslToRgb(h: number, s: number, l: number): [number, number, number]

// Obtener color para una iteración
export function getColor(n: number, maxIterations: number, palette: PaletteType)
```

8. Algoritmos Implementados

8.1 Algoritmo de Mandelbrot

```
function calculateMandelbrot(  
  width: number,  
  height: number,  
  iterations: number,  
  zoom: number,  
  centerX: number,  
  centerY: number,  
  palette: string  
) : ImageData {  
  const imgData = new ImageData(width, height);  
  const data = imgData.data;  
  const colors = buildPalette(palette, iterations);  
  const scale = 1.0 / (width * 0.5 * zoom);  
  
  for (let y = 0; y < height; y++) {  
    const cy = centerY + (y - height/2) * scale;  
  
    for (let x = 0; x < width; x++) {  
      const cx = centerX + (x - width/2) * scale;  
      let zx = 0, zy = 0, n = 0;  
  
      // Iteración  $z = z^2 + c$   
      while (n < iterations) {  
        const zx2 = zx * zx;  
        const zy2 = zy * zy;  
        if (zx2 + zy2 > 4) break; // Radio de escape  
        zy = 2 * zx * zy + cy;  
        zx = zx2 - zy2 + cx;  
        n++;  
      }  
  
      // Asignar color basado en iteraciones de escape  
      const i = (y * width + x) * 4;  
      const colorIdx = n * 3;  
      data[i] = colors[colorIdx];  
      data[i + 1] = colors[colorIdx + 1];  
      data[i + 2] = colors[colorIdx + 2];  
      data[i + 3] = 255;  
    }  
  }  
  
  return imgData;  
}
```

8.2 Algoritmo de Julia

```
function calculateJulia(
  width: number, height: number,
  iterations: number, zoom: number,
  centerX: number, centerY: number,
  cReal: number, cImag: number,
  palette: string
): ImageData {
  const scale = 3.0 / (width * zoom);

  for (let y = 0; y < height; y++) {
    for (let x = 0; x < width; x++) {
      // En Julia, z inicia en el punto del pixel
      let zx = centerX + (x - width/2) * scale;
      let zy = centerY + (y - height/2) * scale;
      let n = 0;

      while (n < iterations) {
        const zx2 = zx * zx;
        const zy2 = zy * zy;
        if (zx2 + zy2 > 4) break;
        // c es constante (parámetro)
        zy = 2 * zx * zy + cImag;
        zx = zx2 - zy2 + cReal;
        n++;
      }
      // ... asignar color
    }
  }
}
```

8.3 Algoritmo de Burning Ship

```
while (n < iterations) {
  const zx2 = zx * zx;
  const zy2 = zy * zy;
  if (zx2 + zy2 > 4) break;

  // Valor absoluto antes de la operación
  const absZx = Math.abs(zx);
  const absZy = Math.abs(zy);
  zy = 2 * absZx * absZy + cy;
  zx = absZx * absZx - absZy * absZy + cx;
  n++;
}
```

8.4 Algoritmo de Tricorn

```
while (n < iterations) {  
  const zx2 = zx * zx;  
  const zy2 = zy * zy;  
  if (zx2 + zy2 > 4) break;  
  
  // Conjugado: signo negativo en parte imaginaria  
  const newZx = zx2 - zy2 + cx;  
  zy = -2 * zx * zy + cy; // Nota: signo negativo  
  zx = newZx;  
  n++;  
}
```

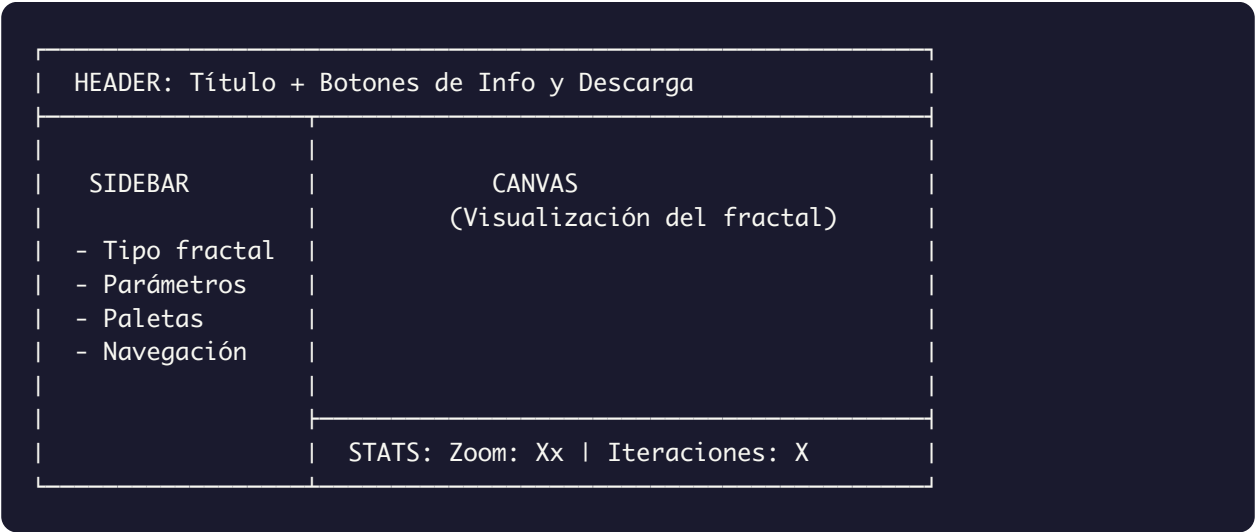
9. Interfaz de Usuario

9.1 Diseño Visual

La interfaz utiliza un esquema de colores oscuro con acentos en verde lima (`lime-400`):

- **Fondo:** Neutral oscuro (`neutral-950` , `neutral-900`)
- **Texto primario:** Blanco
- **Acentos:** Verde lima (`lime-400`)
- **Acentos secundarios:** Púrpura (`purple-400`)
- **Bordes:** Gris (`gray-700` , `gray-800`)

9.2 Layout



9.3 Paletas de Colores Disponibles

Paleta	Descripción
Grayscale	Escala de grises clásica
Fire	Gradiente de negro a amarillo/rojo
Ocean	Gradiente de negro a cian/azul
Rainbow	Espectro completo de colores
Neon	Colores vibrantes neón (cian/magenta)

10. Manual de Instalación

10.1 Requisitos Previos

- **Node.js:** Versión 16 o superior
- **npm** o **yarn** como gestor de paquetes
- Navegador moderno (Chrome, Firefox, Safari, Edge)

10.2 Pasos de Instalación

1. Clonar el repositorio:

```
git clone https://github.com/DaniVegaM/fractal-explorer.git
cd fractal-explorer
```

1. Instalar dependencias:

```
npm install
```

1. Iniciar en modo desarrollo:

```
npm run dev
```

1. Abrir en el navegador:

```
http://localhost:5173
```

10.3 Comandos Disponibles

Comando	Descripción
<code>npm run dev</code>	Inicia el servidor de desarrollo
<code>npm run build</code>	Compila el proyecto para producción
<code>npm run preview</code>	Previsualiza la build de producción
<code>npm run lint</code>	Ejecuta ESLint

10.4 Build de Producción

```
npm run build
```

Los archivos compilados se generarán en la carpeta `dist/`.

11. Manual de Usuario

11.1 Navegación Básica

Acción	Resultado
Click y arrastrar	Mueve la vista por el fractal
Rueda del ratón	Zoom in/out
Botones +/-	Zoom controlado

11.2 Panel de Controles

1. **Tipo de Fractal:** Selecciona entre Mandelbrot, Julia, Burning Ship o Tricorn
2. **Parámetros:**
 - **Iteraciones:** Más iteraciones = más detalle (pero más lento)
 - **Zoom:** Nivel de acercamiento
 - **C Real/Imaginario:** (Solo Julia) Define la constante compleja
3. **Paleta de Colores:** Click en cualquier cuadro de color
4. **Navegación:** Botones para zoom y reiniciar vista

11.3 Exportar Imagen

1. Click en el ícono de descarga (↓) en el header
2. Se descargará automáticamente un archivo PNG con el formato:

```
{fractal}_z{zoom}_{paleta}_{timestamp}.png
```

11.4 Panel de Información

Click en el ícono (i) para ver:

- Descripción matemática del fractal actual
 - Guía de controles
 - Información del desarrollador
-

12. Optimizaciones de Rendimiento

12.1 Web Workers

Los cálculos de fractal se ejecutan en un hilo separado mediante Web Workers, lo que permite:

- **No bloquear la UI** durante cálculos intensivos
- **Mantener la responsividad** de la interfaz

12.2 Renderizado Progresivo

Sistema de dos fases:

1. **Baja resolución:** Durante interacción (arrastre, zoom) se renderiza a 35% de resolución
2. **Alta resolución:** Después de 250ms sin interacción, se renderiza a resolución completa

```
const renderToCanvas = (params, center, lowRes = false) => {
  const scale = lowRes ? 0.35 : 1;
  const renderWidth = Math.floor(canvas.width * scale);
  // ...
};
```

12.3 Caché de Paletas

Las paletas de colores se precálculan y almacenan en caché:

```
const PALETTE_CACHE: Record<string, Uint8Array> = {};

function buildPalette(palette: string, maxIter: number): Uint8Array {
  const key = `${palette}-${maxIter}`;
  if (PALETTE_CACHE[key]) return PALETTE_CACHE[key];
  // ... calcular y cachear
}
```

12.4 Transferencia de Datos Optimizada

Uso de **Transferable Objects** para transferir datos entre Worker y main thread sin copia:

```
self.postMessage(
  { type: 'result', data: imageData.data, width, height },
  [imageData.data.buffer] // Transferable
);
```

12.5 Canvas Offscreen

Para evitar parpadeos durante el renderizado de baja resolución:

```
const offscreenCanvasRef = useRef<HTMLCanvasElement | null>(null);  
// Se usa para escalar la imagen de baja res
```

13. Conclusiones

13.1 Logros del Proyecto

1. **Implementación exitosa** de cuatro tipos de fractales con sus algoritmos matemáticos correspondientes.
2. **Interfaz de usuario intuitiva** que permite exploración fluida de los fractales.
3. **Rendimiento optimizado** mediante Web Workers y renderizado progresivo.
4. **Arquitectura modular** que facilita la extensión y mantenimiento del código.
5. **Personalización visual** con múltiples paletas de colores.

13.2 Posibles Mejoras Futuras

- Implementar más tipos de fractales (Newton, Barnsley Fern, etc.)
- Añadir smooth coloring para suavizar bandas de color
- Implementar zoom semántico (zoom hacia el cursor)
- Añadir animaciones de parámetros
- Guardar y cargar coordenadas de ubicaciones interesantes
- Modo de renderizado de alta resolución para exportación

13.3 Aprendizajes

El desarrollo de este proyecto permitió profundizar en:

- Matemáticas de números complejos y fractales
- Optimización de rendimiento en aplicaciones web
- Uso avanzado de Web Workers y Canvas API
- Arquitectura de aplicaciones React con TypeScript

14. Referencias

14.1 Bibliografía

1. Mandelbrot, B. B. (1982). *The Fractal Geometry of Nature*. W. H. Freeman and Company.
2. Peitgen, H.-O., Jürgens, H., & Saupe, D. (2004). *Chaos and Fractals: New Frontiers of Science*. Springer.
3. Devaney, R. L. (1989). *An Introduction to Chaotic Dynamical Systems*. Addison-Wesley.

14.2 Recursos en Línea

- **MDN Web Docs - Canvas API:** https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
- **MDN Web Docs - Web Workers:** https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API
- **React Documentation:** <https://react.dev/>
- **TypeScript Handbook:** <https://www.typescriptlang.org/docs/>
- **Vite Documentation:** <https://vitejs.dev/>
- **TailwindCSS Documentation:** <https://tailwindcss.com/docs>

14.3 Herramientas

- **Visual Studio Code:** Editor de código
 - **Git & GitHub:** Control de versiones
 - **npm:** Gestor de paquetes
 - **Chrome DevTools:** Depuración y análisis de rendimiento
-

Anexos

Anexo A: Fórmulas Matemáticas

Fractal	Fórmula	Radio de Escape
Mandelbrot	$z_{n+1} = z_n^2 + c$	$ z > 2$
Julia	$z_{n+1} = z_n^2 + c$ (c constante)	$ z > 2$
Burning Ship	$z_{n+1} = (\operatorname{Re}(z_n) + i \operatorname{Im}(z_n))^2 + c$	$ z > 2$
Tricorn	$z_{n+1} = \bar{z}_n^2 + c$	$ z > 2$

Anexo B: Configuración de Vite

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react-swc'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [
    react(),
    tailwindcss()
  ],
  worker: {
    format: 'es',
  },
})
```

Anexo C: Estructura de Tipos TypeScript

```
// Tipos de fractal disponibles
export type FractalType = "mandelbrot" | "julia" | "burning-ship" | "tricorn";

// Paletas de colores disponibles
export type PaletteType =
  | 'grayscale' | 'fire' | 'ocean' | 'cosmic'
  | 'rainbow' | 'neon' | 'earth' | 'gradient';

// Parámetro individual de fractal
export interface FractalParam {
  label: string;
  key: string;
  min: number;
  max: number;
  step: number;
  default: number;
  description: string;
}

// Configuración completa de un fractal
export interface FractalConfig {
  name: string;
  description: string;
  params: FractalParam[];
  defaultCenter: { x: number; y: number };
}
```